Assignment Tracking Application

1. Problem Statement

Develop a RESTful **Assignment Tracking Application** using Spring Boot that allows an organization/training institute to manage trainers, students, assignments, submissions, and evaluation.

The application should have four logical roles:

- **Super Admin** – manages administrators and overall system data.
- **Admin** – manages trainers, students, and assignments.
- **Trainer** – creates/manages assignments and evaluates student submissions.
- **Student** – views assigned assignments and submits their work.

---

2. Technology Requirements

Technology	Purpose
Java 17+	Programming language
Spring Boot	Application framework
Spring Web	REST APIs
Spring Data JPA	Database persistence
MySQL	Relational database
Lombok	Reduce boilerplate
Jakarta Validation	Request validation
Postman/Swagger	API testing

---

3. Main Features

Super Admin

The Super Admin should be able to:

- Create administrators.
- View all administrators.
- Update administrator details.
- Delete/deactivate administrators.
- View trainers and students.
- View all assignments.
- View all submissions.
- View overall assignment statistics.

Admin

The Admin should be able to:

- Create trainers.
- Create students.
- Update trainer/student information.
- View trainers.
- View students.
- Create assignments.
- Assign assignments to students.
- View assignment details.
- View student submissions.
- Delete assignments where appropriate.

Trainer

The Trainer should be able to:

- View assigned/created assignments.
- Create assignments.
- Update assignments.
- Assign assignments to students.
- View students.
- View student submissions.
- Evaluate submissions.
- Give marks and feedback.
- Change submission status.

Student

The Student should be able to:

- View their profile.
- View assignments assigned to them.
- View assignment details.
- Submit an assignment.
- Update their submission before the deadline.
- View submission status.
- View marks and trainer feedback.

---

4. Suggested Database Entities

You can implement the following entities:

1. User

Common information for Admin, Trainer, and Student.

User
---
id
name
email
phone
role
active
createdAt
updatedAt

`role` can contain:

SUPER_ADMIN
ADMIN
TRAINER
STUDENT

2. Assignment

Assignment
---

id
title
description
assignedDate
dueDate
maxMarks
status
trainerId
createdAt
updatedAt

Possible status:

CREATED
ASSIGNED
CLOSED

3. AssignmentStudent

This represents which students received which assignments.

AssignmentStudent
---
id
assignmentId
studentId
assignedAt

This is useful because one assignment can be assigned to multiple students.

4. Submission

Submission
---
id
assignmentId
studentId
submissionText
submittedAt
marks
feedback
status
evaluatedAt

Possible status:

PENDING
SUBMITTED
EVALUATED
LATE

---

5. API Endpoints

A. Super Admin APIs

Create Admin

POST /api/super-admin/admins

Request:

```
{
  "name": "John Admin",
  "email": "john@example.com",
  "phone": "9876543210"
}
```

Response:

```
{
  "id": 1,
  "name": "John Admin",
  "email": "john@example.com",
  "phone": "9876543210",
  "role": "ADMIN",
  "active": true
}
```

Get All Admins

GET /api/super-admin/admins

Get Admin By ID

GET /api/super-admin/admins/{id}

Update Admin

PUT /api/super-admin/admins/{id}

Delete Admin

DELETE /api/super-admin/admins/{id}

Get All Trainers

GET /api/super-admin/trainers

Get All Students

GET /api/super-admin/students

Get All Assignments

GET /api/super-admin/assignments

Get All Submissions

GET /api/super-admin/submissions

Dashboard Statistics

GET /api/super-admin/dashboard

Example:


```
{
  "totalAdmins": 3,
  "totalTrainers": 10,
  "totalStudents": 100,
  "totalAssignments": 50,
  "totalSubmissions": 400,
  "evaluatedSubmissions": 350,
  "pendingSubmissions": 50
}
```


---

7. Admin APIs

Trainer Management

Create Trainer

POST /api/admin/trainers

Request:

```
{
  "name": "Robert",
  "email": "robert@example.com",
  "phone": "9876543211"
}
```

Get All Trainers

GET /api/admin/trainers

Get Trainer

GET /api/admin/trainers/{id}

Update Trainer

PUT /api/admin/trainers/{id}

Delete Trainer

DELETE /api/admin/trainers/{id}

Student Management

Create Student

POST /api/admin/students

Request:

```
{
  "name": "Alice",
  "email": "alice@example.com",
  "phone": "9876543212"
}
```

Get All Students

GET /api/admin/students

Get Student

GET /api/admin/students/{id}

Update Student

PUT /api/admin/students/{id}

Delete Student

DELETE /api/admin/students/{id}

---

8. Admin Assignment APIs

Create Assignment

POST /api/admin/assignments

Request:

```
{
  "title": "Spring Boot CRUD Application",
  "description": "Develop a CRUD application using Spring Boot and JPA",
  "dueDate": "2026-09-01",
  "maxMarks": 100,
  "trainerId": 10
}
```

Get All Assignments

GET /api/admin/assignments

Get Assignment

GET /api/admin/assignments/{id}

Update Assignment

PUT /api/admin/assignments/{id}

Delete Assignment

DELETE /api/admin/assignments/{id}

Assign Assignment to Students

POST /api/admin/assignments/{assignmentId}/students

Request:

```
{
  "studentIds": [101, 102, 103]
}
```

View Submissions

GET /api/admin/assignments/{assignmentId}/submissions

---

9. Trainer APIs

Assignment Management

Create Assignment

POST /api/trainers/{trainerId}/assignments

Request:

```
{
  "title": "JPA Relationship Assignment",
  "description": "Implement One-to-Many and Many-to-One relationships",
  "dueDate": "2026-09-10",
  "maxMarks": 50
}
```

Get Trainer's Assignments

GET /api/trainers/{trainerId}/assignments

Get Assignment Details

GET /api/trainers/{trainerId}/assignments/{assignmentId}

Update Assignment

PUT /api/trainers/{trainerId}/assignments/{assignmentId}

Delete Assignment

DELETE /api/trainers/{trainerId}/assignments/{assignmentId}

Assign Students

Assign Assignment

POST /api/trainers/{trainerId}/assignments/{assignmentId}/students

Request:

```
{
  "studentIds": [101, 102, 103]
}
```

View Assigned Students

GET /api/trainers/{trainerId}/assignments/{assignmentId}/students

---

Submission Evaluation

View Submissions

GET /api/trainers/{trainerId}/submissions

View Assignment Submissions

GET /api/trainers/{trainerId}/assignments/{assignmentId}/submissions

Evaluate Submission

PUT /api/trainers/{trainerId}/submissions/{submissionId}/evaluate

Request:

```
{
  "marks": 85,
  "feedback": "Good implementation. Improve exception handling."
}
```

Response:

```
{
  "id": 501,
  "assignmentId": 20,
  "studentId": 101,
  "marks": 85,
  "feedback": "Good implementation. Improve exception handling.",
  "status": "EVALUATED"
}
```

---

10. Student APIs

Profile

Get Student Profile

GET /api/students/{studentId}

Update Profile

PUT /api/students/{studentId}

Assignment APIs

View My Assignments

GET /api/students/{studentId}/assignments

View Assignment Details

GET /api/students/{studentId}/assignments/{assignmentId}

Example response:


```
{
  "id": 20,
  "title": "Spring Boot CRUD Application",
  "description": "Develop a CRUD application using Spring Boot",
  "assignedDate": "2026-08-20",
  "dueDate": "2026-09-01",
  "maxMarks": 100,
  "status": "ASSIGNED"
}
```


---

11. Student Submission APIs

Submit Assignment

POST /api/students/{studentId}/assignments/{assignmentId}/submissions

Request:


```
{
  "submissionText": "https://github.com/student/springboot-assignment"
}
```

View My Submission

GET /api/students/{studentId}/assignments/{assignmentId}/submission

Update Submission

PUT /api/students/{studentId}/assignments/{assignmentId}/submission

Request:

```
{
  "submissionText": "https://github.com/student/final-assignment"
}
```

View All My Submissions

GET /api/students/{studentId}/submissions

View Result

GET /api/students/{studentId}/submissions/{submissionId}/result

Example:

```
{
  "submissionId": 501,
  "assignmentTitle": "Spring Boot CRUD Application",
  "marks": 85,
  "maxMarks": 100,
  "feedback": "Good implementation.",
  "status": "EVALUATED"
}
```

---

12. Validation Requirements

Use Jakarta Validation annotations.

For example:

```
@NotBlank
private String name;

@NotBlank
@Email
private String email;
```

```
@NotBlank
@Size(min = 10, max = 10)
private String phone;
```

Assignment:

```
@NotBlank
@Size(min = 5, max = 100)
private String title;

@NotBlank
private String description;

@NotNull
@Future
private LocalDate dueDate;

@NotNull
@Min(1)
private Integer maxMarks;
```

Submission:

```
@NotBlank
private String submissionText;
```

For evaluation:

```
@NotNull
@Min(0)
private Integer marks;

@NotBlank
private String feedback;
```

You should also validate that:

- Email is unique.
- Student exists before assigning an assignment.
- Trainer exists before creating an assignment.
- Assignment exists before submission.
- Student is actually assigned to the assignment.
- Marks cannot exceed `maxMarks`.

- A student cannot submit the same assignment multiple times unless update is explicitly allowed.
- Submission cannot be modified after evaluation.
- Assignment due date cannot be in the past when creating it.

---

13. Exception Handling

Implement a global exception handler using:

@RestControllerAdvice

Handle at least:

ResourceNotFoundException
DuplicateResourceException
InvalidAssignmentException
InvalidSubmissionException
MethodArgumentNotValidException

Example error response:

```
{
  "timestamp": "2026-08-21T10:30:00",
  "status": 404,
  "error": "NOT_FOUND",
  "message": "Student with id 101 not found"
}
```

---

14. Suggested Package Structure

```
com.example.assignmenttracker
│
├── controller
│   ├── SuperAdminController
│   ├── AdminController
│   ├── TrainerController
│   └── StudentController
│
├── service
│   ├── UserService
```

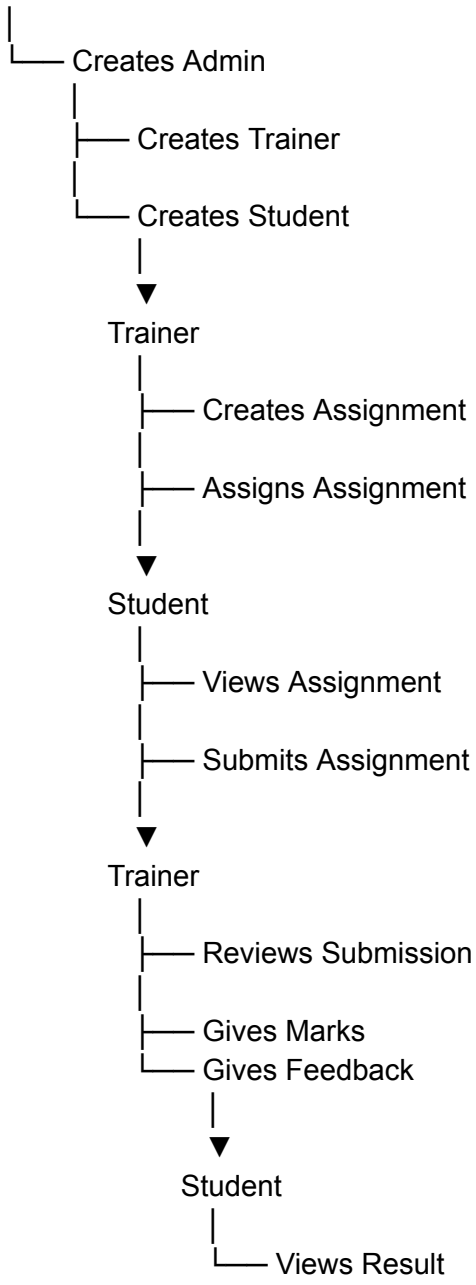
```
│   ├── AssignmentService
│   ├── SubmissionService
│   └── DashboardService
│
├── service.impl
│   ├── UserServiceImpl
│   ├── AssignmentServiceImpl
│   └── SubmissionServiceImpl
│
├── repository
│   ├── UserRepository
│   ├── AssignmentRepository
│   ├── AssignmentStudentRepository
│   └── SubmissionRepository
│
├── entity
│   ├── User
│   ├── Assignment
│   ├── AssignmentStudent
│   └── Submission
│
├── dto
│   ├── UserRequest
│   ├── UserResponse
│   ├── AssignmentRequest
│   ├── AssignmentResponse
│   ├── SubmissionRequest
│   ├── EvaluationRequest
│   └── DashboardResponse
│
├── exception
│   ├── ResourceNotFoundException
│   ├── DuplicateResourceException
│   └── GlobalExceptionHandler
│
└── AssignmentTrackerApplication
```

---

15. Important Business Flow

The expected workflow is:

Super Admin



---

16. Mini Assignment Expectations

The candidate should demonstrate:

Spring Boot

- Project configuration
- REST controllers

- Dependency injection
- Configuration through `application.properties`

Spring Web

- `@RestController`
- `@RequestMapping`
- `@GetMapping`
- `@PostMapping`
- `@PutMapping`
- `@DeleteMapping`
- `@PathVariable`
- `@RequestBody`

JPA

- `@Entity`
- `@Id`
- `@GeneratedValue`
- `@ManyToOne`
- `@OneToMany` where appropriate
- Repository interfaces
- JPQL/derived queries

Validation

- `@Valid`
- `@NotBlank`
- `@NotNull`
- `@Email`
- `@Min`
- `@Future`
- `@Size`

Lombok

Use:

@Getter
@Setter
@NoArgsConstructor

@AllArgsConstructor
@Builder

where appropriate.

---

17. Additional Features

- Pagination for students, assignments, and submissions.
- Search students by name/email.
- Filter assignments by status.
- Filter submissions by `PENDING`, `SUBMITTED`, `EVALUATED`, etc.
- Sorting by due date.
- Dashboard statistics.
- Assignment completion percentage.
- Average marks per assignment.
- Automatic `LATE` status when submission occurs after the due date.
- DTO mapping instead of exposing entities directly.